NATURAL LANGUAGE PROCESSING LABORATORY

LAB RECORD

DSA0303 - NATURAL LANGUAGE PROCESSING

Dec 2026 (UG) • SLOT A

Implementation of Fundamental NLP Techniques using Python (Google Colab)

Name	: B Tharun
Register Number	: 1924260010
Department	: AIDS
Year	: 1st
Subject Code	: DSA0303
Faculty	: Akshaya K

INDEX

Ex. No.	Date	Name of the Experiment	Page No.	Signature
1		Tokenization (Sentence and Word Level)	3	
2		Stop Word Removal	5	
3		Stemming (Porter, Lancaster and Snowball Stemmers)	7	
4		Lemmatization	9	
5		Part-of-Speech (POS) Tagging	11	
6		Named Entity Recognition (NER)	13	
7		N-gram Generation and Bigram Probability	15	
8		Bag of Words (BoW) Representation	17	
9		TF-IDF (Term Frequency - Inverse Document Frequency)	19	
10		Sentiment Analysis	22	

EXPERIMENT NO: 1

TOKENIZATION (SENTENCE AND WORD LEVEL)

Date: _____

AIM:

To perform tokenization on a text paragraph by splitting it into sentences and words using the sentence, Treebank, WordPunct, regular-expression and Tweet tokenizers of the NLTK library, and to compare how each tokenizer treats punctuation and contractions.

ALGORITHM:

Step 1: Start the program.

Step 2: Import PunktSentenceTokenizer, TreebankWordTokenizer, WordPunctTokenizer, RegexpTokenizer and TweetTokenizer from nltk.tokenize.

Step 3: Store the input paragraph in the variable text.

Step 4: Create a PunktSentenceTokenizer object and call tokenize() to split the paragraph into sentences; print the number of sentences and each sentence.

Step 5: Select the third sentence, which contains a contraction and a question mark, as the sample for word tokenization.

Step 6: Tokenize the sample using simple whitespace split(), TreebankWordTokenizer, WordPunctTokenizer, RegexpTokenizer(r"\w+") and TweetTokenizer, and print each token list.

Step 7: Tokenize the first sentence into words with the Treebank tokenizer and count the tokens.

Step 8: Compare the outputs of all tokenizers.

Step 9: Stop the program.

CODE:

```
from nltk.tokenize import (PunktSentenceTokenizer, TreebankWordTokenizer,
                           WordPunctTokenizer, RegexpTokenizer, TweetTokenizer)

text = ("Natural Language Processing helps computers understand human language. "
        "It is used in chatbots, search engines and translators! "
        "Isn't it amazing?")

# 1. Sentence tokenization
sentences = PunktSentenceTokenizer().tokenize(text)
print("Number of sentences:", len(sentences))
for i, s in enumerate(sentences, 1):
    print(f"  Sentence {i}: {s}")

# 2. Word tokenization using different tokenizers
sample = sentences[2]
print("\nSample sentence :", sample)
print("Whitespace split :", sample.split())
print("Treebank         :", TreebankWordTokenizer().tokenize(sample))
print("WordPunct        :", WordPunctTokenizer().tokenize(sample))
print("Regexp (words)   :", RegexpTokenizer(r"\w+").tokenize(sample))
print("Tweet            :", TweetTokenizer().tokenize(sample))

# 3. Word tokens of the first sentence
words = TreebankWordTokenizer().tokenize(sentences[0])
print("\nWords in sentence 1 :", words)
print("Total word tokens   :", len(words))
```

✓0s

OUTPUT:

```
Number of sentences: 3
  Sentence 1: Natural Language Processing helps computers understand human language.
  Sentence 2: It is used in chatbots, search engines and translators!
  Sentence 3: Isn't it amazing?

Sample sentence : Isn't it amazing?
Whitespace split : ["Isn't", 'it', 'amazing?']
Treebank         : ['Is', "n't", 'it', 'amazing', '?']
WordPunct        : ['Isn', "'", 't', 'it', 'amazing', '?']
Regexp (words)   : ['Isn', 't', 'it', 'amazing']
Tweet            : ["Isn't", 'it', 'amazing', '?']

Words in sentence 1 : ['Natural', 'Language', 'Processing', 'helps', 'computers', 'understand',
     'human', 'language', '.']
Total word tokens   : 9
```

RESULT:

The paragraph was split into 3 sentences, and the first sentence produced 9 word tokens (including the full stop). For the sample sentence "Isn't it amazing?", the Treebank tokenizer separated the contraction into "Is" and "n't", the WordPunct tokenizer broke it into "Isn", "'" and "t", the regular-expression tokenizer discarded the punctuation, and whitespace splitting left "?" attached to "amazing". Thus tokenization was implemented successfully, and the choice of tokenizer was found to affect the tokens passed to later NLP stages.

EXPERIMENT NO: 2

STOP WORD REMOVAL

Date: _____

AIM:

To remove stop words (high-frequency words such as "the", "is" and "in" that carry little meaning) from a text using the English stop word list, and to measure how much the text is reduced.

ALGORITHM:

Step 1: Start the program.

Step 2: Import the re module and the STOP_WORDS set from spacy.lang.en.stop_words.

Step 3: Store the input sentence in the variable text.

Step 4: Convert the text to lower case and extract the alphabetic words using the regular expression [A-Za-z]+ to obtain the tokens.

Step 5: For every token, check whether it is present in the STOP_WORDS set.

Step 6: Collect the tokens that are present in the set as removed words, and the remaining tokens as filtered tokens.

Step 7: Print the size of the stop word list, the original tokens, the stop words found and the filtered tokens.

Step 8: Print the number of tokens before and after removal.

Step 9: Stop the program.

CODE:

```
import re
from spacy.lang.en.stop_words import STOP_WORDS

text = ("This is a sample sentence, showing off the stop words "
        "filtration in Natural Language Processing.")

# Tokenize and convert to lower case
tokens = re.findall(r"[A-Za-z]+", text.lower())

# Separate stop words and content words
removed  = [w for w in tokens if w in STOP_WORDS]
filtered = [w for w in tokens if w not in STOP_WORDS]

print("Size of English stop word list :", len(STOP_WORDS))
print("Original text   :", text)
print("Original tokens :", tokens)
print("Stop words found:", removed)
print("Filtered tokens :", filtered)
print("Tokens before / after removal :", len(tokens), "/", len(filtered))
```

✓0s

OUTPUT:

```
Size of English stop word list : 326
Original text   : This is a sample sentence, showing off the stop words filtration in Natural
    Language Processing.
Original tokens : ['this', 'is', 'a', 'sample', 'sentence', 'showing', 'off', 'the', 'stop',
    'words', 'filtration', 'in', 'natural', 'language', 'processing']
Stop words found: ['this', 'is', 'a', 'off', 'the', 'in']
Filtered tokens : ['sample', 'sentence', 'showing', 'stop', 'words', 'filtration', 'natural',
    'language', 'processing']
Tokens before / after removal : 15 / 9
```

RESULT:

The English stop word list used contained 326 words. The sentence had 15 tokens; six stop words (this, is, a, off, the, in) were removed, leaving 9 meaningful tokens: sample, sentence, showing, stop, words, filtration, natural, language and processing. Thus stop word removal was performed successfully and reduced the text size by 40 percent.

EXPERIMENT NO: 3

STEMMING (PORTER, LANCASTER AND SNOWBALL STEMMERS)

Date: _____

AIM:

To reduce words to their stem (root form) using the Porter, Lancaster and Snowball stemming algorithms of NLTK, and to compare the output of the three stemmers.

ALGORITHM:

Step 1: Start the program.

Step 2: Import PorterStemmer, LancasterStemmer and SnowballStemmer from nltk.stem.

Step 3: Create a list of sample words containing different inflections and derivations.

Step 4: Create one object each of PorterStemmer, LancasterStemmer and SnowballStemmer (language = English).

Step 5: For every word in the list, call the stem() method of the three stemmers.

Step 6: Print the word and its three stems in a tabular format.

Step 7: Take a sentence, split it into words and apply the Porter stemmer to every word.

Step 8: Join the stemmed words and print the stemmed sentence.

Step 9: Stop the program.

CODE:

```
from nltk.stem import PorterStemmer, LancasterStemmer, SnowballStemmer

words = ["running", "runs", "easily", "fairly", "studies", "studying",
         "generously", "happiness", "organization", "better"]

porter    = PorterStemmer()
lancaster = LancasterStemmer()
snowball  = SnowballStemmer("english")

print(f"{'Word':<15}{'Porter':<15}{'Lancaster':<15}{'Snowball':<15}")
print("-" * 60)
for w in words:
    print(f"{w:<15}{porter.stem(w):<15}{lancaster.stem(w):<15}"
          f"{snowball.stem(w):<15}")

sentence = "The children were playing happily and studying hard"
print("\nOriginal sentence :", sentence)
print("Porter stemmed    :", " ".join(porter.stem(w) for w in sentence.split()))
```

✓0s

OUTPUT:

```
Word           Porter         Lancaster      Snowball
-------------------------------------------------------------
running        run            run            run
runs           run            run            run
easily         easili         easy           easili
fairly         fairli         fair           fair
studies        studi          study          studi
studying       studi          study          studi
generously     gener          gen            generous
happiness      happi          happy          happi
organization   organ          org            organ
better         better         bet            better

Original sentence : The children were playing happily and studying hard
Porter stemmed    : the children were play happili and studi hard
```

RESULT:

Words such as running and runs were reduced to the stem "run" by all three stemmers. The Lancaster stemmer was the most aggressive (organization became "org", generously became "gen" and better became "bet"), the Snowball stemmer kept longer stems (generously became "generous"), and the Porter stemmer produced stems that are not dictionary words (easili, studi, happi). Thus stemming was implemented successfully, and it was observed that stemming is fast but may lose meaning.

EXPERIMENT NO: 4

LEMMATIZATION

Date: _____

AIM:

To convert words to their dictionary base form (lemma) using the spaCy library, and to compare lemmatization with Porter stemming.

ALGORITHM:

Step 1: Start the program.

Step 2: Import spacy and PorterStemmer, and load the English pipeline en_core_web_sm using spacy.load().

Step 3: Store the input sentence in the variable text.

Step 4: Pass the text to the pipeline nlp(text) to obtain a Doc object; the pipeline tokenizes the text, tags the part of speech of every token and assigns a lemma.

Step 5: For every token in the Doc, read token.text, token.pos_ and token.lemma_.

Step 6: Apply the Porter stemmer to the same token for comparison.

Step 7: Print the token, POS tag, lemma and stem in a tabular format.

Step 8: Join all lemmas to print the lemmatized sentence.

Step 9: Stop the program.

CODE:

```
# Run once in Google Colab:  !python -m spacy download en_core_web_sm
import spacy
from nltk.stem import PorterStemmer

nlp = spacy.load("en_core_web_sm")
porter = PorterStemmer()

text = "The striped bats were hanging on their feet and eating better leaves"
doc = nlp(text)

print(f"{'Token':<10}{'POS':<8}{'Lemma':<10}{'Porter Stem':<12}")
print("-" * 40)
for token in doc:
    print(f"{token.text:<10}{token.pos_:<8}{token.lemma_:<10}"
          f"{porter.stem(token.text):<12}")

print("\nOriginal   :", text)
print("Lemmatized :", " ".join(t.lemma_ for t in doc))
```

✓0s

OUTPUT:

```
Token     POS     Lemma     Porter Stem
--------------------------------------
The       DET     the       the
striped   ADJ     striped   stripe
bats      NOUN    bat       bat
were      AUX     be        were
hanging   VERB    hang      hang
on        ADP     on        on
their     PRON    their     their
feet      NOUN    foot      feet
and       CCONJ   and       and
eating    VERB    eat       eat
better    ADJ     well      better
leaves    NOUN    leaf      leav

Original   : The striped bats were hanging on their feet and eating better leaves
Lemmatized : the striped bat be hang on their foot and eat well leaf
```

RESULT:

Lemmatization converted the words to valid dictionary forms using their part of speech: bats became "bat", feet became "foot", leaves became "leaf", were became "be", hanging became "hang" and eating became "eat". In comparison, the Porter stemmer left "feet" and "were" unchanged and reduced "leaves" to the non-word "leav". Thus lemmatization gives more meaningful base words than stemming.

EXPERIMENT NO: 5

PART-OF-SPEECH (POS) TAGGING

Date: _____

AIM:

To assign a part-of-speech tag (noun, verb, adjective, etc.) to every word of a sentence using the spaCy statistical tagger, and to find the frequency of each tag.

ALGORITHM:

Step 1: Start the program.

Step 2: Import spacy and Counter from the collections module.

Step 3: Load the English pipeline en_core_web_sm.

Step 4: Store the input sentence in the variable text and process it with nlp(text) to get a Doc object.

Step 5: For every token in the Doc, read the coarse tag token.pos_ and the fine-grained Penn Treebank tag token.tag_.

Step 6: Use spacy.explain() to obtain the meaning of each fine-grained tag.

Step 7: Print the token, POS, tag and meaning in a tabular format.

Step 8: Count the occurrences of every POS tag using Counter and print the frequency.

Step 9: Stop the program.

CODE:

```
import spacy
from collections import Counter

nlp = spacy.load("en_core_web_sm")

text = "The quick brown fox jumps over the lazy dog near the river bank."
doc = nlp(text)

print(f"{'Token':<9}{'POS':<7}{'Tag':<6}{'Meaning of Tag'}")
print("-" * 62)
for t in doc:
    print(f"{t.text:<9}{t.pos_:<7}{t.tag_:<6}{spacy.explain(t.tag_)}")

print("\nPOS frequency:", dict(Counter(t.pos_ for t in doc)))
```

✓0s

OUTPUT:

```
Token    POS    Tag   Meaning of Tag
-----------------------------------------------------------
The      DET    DT    determiner
quick    ADJ    JJ    adjective (English), other noun-modifier (Chinese)
brown    ADJ    JJ    adjective (English), other noun-modifier (Chinese)
fox      NOUN   NN    noun, singular or mass
jumps    VERB   VBZ   verb, 3rd person singular present
over     ADP    IN    conjunction, subordinating or preposition
the      DET    DT    determiner
lazy     ADJ    JJ    adjective (English), other noun-modifier (Chinese)
dog      NOUN   NN    noun, singular or mass
near     ADP    IN    conjunction, subordinating or preposition
the      DET    DT    determiner
river    NOUN   NN    noun, singular or mass
bank     NOUN   NN    noun, singular or mass
.        PUNCT  .     punctuation mark, sentence closer

POS frequency: {'DET': 3, 'ADJ': 3, 'NOUN': 4, 'VERB': 1, 'ADP': 2, 'PUNCT': 1}
```

RESULT:

All 14 tokens of the sentence were tagged. "fox", "dog", "river" and "bank" were tagged NOUN, "quick", "brown" and "lazy" as ADJ, "jumps" as VERB (VBZ), "over" and "near" as ADP, and the full stop as PUNCT. The tag frequency was NOUN 4, DET 3, ADJ 3, ADP 2, VERB 1 and PUNCT 1. Thus POS tagging was performed successfully.

EXPERIMENT NO: 6

NAMED ENTITY RECOGNITION (NER)

Date: _____

AIM:

To identify and classify named entities such as persons, organizations, locations, dates and monetary values in a text using the spaCy named entity recognizer.

ALGORITHM:

Step 1: Start the program.

Step 2: Import spacy and load the English pipeline en_core_web_sm.

Step 3: Store the input text, containing names of people, organizations, places, a date and an amount, in the variable text.

Step 4: Process the text using nlp(text) to create a Doc object.

Step 5: Iterate over doc.ents, which contains the entities detected by the recognizer.

Step 6: For every entity, read the entity text (ent.text) and its label (ent.label_).

Step 7: Use spacy.explain() to get the description of each label.

Step 8: Print the entity, label and description, and print the total number of entities found.

Step 9: Stop the program.

CODE:

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = ("Sundar Pichai, the CEO of Google, met Narendra Modi in New Delhi "
        "on Monday to announce a 10 billion dollar investment in India.")
doc = nlp(text)

print("Text:", text, "\n")
print(f"{'Entity':<18}{'Label':<10}{'Description'}")
print("-" * 60)
for ent in doc.ents:
    print(f"{ent.text:<18}{ent.label_:<10}{spacy.explain(ent.label_)}")

print("\nTotal entities found:", len(doc.ents))
```

✓0s

OUTPUT:

```
Text: Sundar Pichai, the CEO of Google, met Narendra Modi in New Delhi on Monday to announce a
    10 billion dollar investment in India.

Entity            Label     Description
-----------------------------------------------------------
Sundar Pichai     PERSON    People, including fictional
Google            ORG       Companies, agencies, institutions, etc.
Narendra Modi     PERSON    People, including fictional
New Delhi         GPE       Countries, cities, states
Monday            DATE      Absolute or relative dates or periods
10 billion dollar MONEY     Monetary values, including unit
India             GPE       Countries, cities, states

Total entities found: 7
```

RESULT:

Seven named entities were recognized: Sundar Pichai and Narendra Modi (PERSON), Google (ORG), New Delhi and India (GPE), Monday (DATE) and 10 billion dollar (MONEY). Thus named entity recognition was performed successfully, and the unstructured text was converted into labelled information.

EXPERIMENT NO: 7

N-GRAM GENERATION AND BIGRAM PROBABILITY

Date: _____

AIM:

To generate unigrams, bigrams and trigrams from a sentence using NLTK, and to calculate the bigram probability P(w2 | w1) for a simple language model.

ALGORITHM:

Step 1: Start the program.

Step 2: Import Counter from collections and ngrams from nltk.util.

Step 3: Store the input sentence in the variable text and split it into tokens.

Step 4: For n = 1, 2 and 3, call ngrams(tokens, n) to generate the n-grams of the sentence.

Step 5: Join the words of every n-gram and print the list of unigrams, bigrams and trigrams along with their counts.

Step 6: Count the frequency of every word (unigram counts) and every bigram using Counter.

Step 7: For every bigram (w1, w2), calculate P(w2 | w1) = count(w1, w2) / count(w1).

Step 8: Print the probabilities.

Step 9: Stop the program.

CODE:

```
from collections import Counter
from nltk.util import ngrams

text = "natural language processing makes natural language understanding easy"
tokens = text.split()
print("Tokens:", tokens, "\n")

for n, name in [(1, "Unigrams"), (2, "Bigrams"), (3, "Trigrams")]:
    grams = list(ngrams(tokens, n))
    print(f"{name} (count = {len(grams)}):")
    print("  ", [" ".join(g) for g in grams], "\n")

# Bigram probability  P(w2 | w1) = count(w1, w2) / count(w1)
unigram_count = Counter(tokens)
bigram_count = Counter(ngrams(tokens, 2))
print("Bigram probabilities:")
for (w1, w2), c in bigram_count.items():
    print(f"  P({w2} | {w1}) = {c}/{unigram_count[w1]} = {c / unigram_count[w1]:.2f}")
```

✓0s

OUTPUT:

```
Tokens: ['natural', 'language', 'processing', 'makes', 'natural', 'language', 'understanding',
    'easy']

Unigrams (count = 8):
   ['natural', 'language', 'processing', 'makes', 'natural', 'language', 'understanding',
    'easy']

Bigrams (count = 7):
   ['natural language', 'language processing', 'processing makes', 'makes natural', 'natural
    language', 'language understanding', 'understanding easy']

Trigrams (count = 6):
   ['natural language processing', 'language processing makes', 'processing makes natural',
    'makes natural language', 'natural language understanding', 'language understanding easy']

Bigram probabilities:
  P(language | natural) = 2/2 = 1.00
  P(processing | language) = 1/2 = 0.50
  P(makes | processing) = 1/1 = 1.00
  P(natural | makes) = 1/1 = 1.00
  P(understanding | language) = 1/2 = 0.50
  P(easy | understanding) = 1/1 = 1.00
```

RESULT:

From a sentence of 8 tokens, 8 unigrams, 7 bigrams and 6 trigrams were generated, which agrees with the rule that N tokens give N - n + 1 n-grams. The bigram "natural language" occurs twice, so P(language | natural) = 1.00, while P(processing | language) = P(understanding | language) = 0.50. Thus n-grams and bigram probabilities were calculated successfully.

EXPERIMENT NO: 8

BAG OF WORDS (BOW) REPRESENTATION

Date: _____

AIM:

To convert a collection of text documents into numerical feature vectors using the Bag of Words model implemented by CountVectorizer of scikit-learn.

ALGORITHM:

Step 1: Start the program.

Step 2: Import pandas and CountVectorizer from sklearn.feature_extraction.text.

Step 3: Create a list of three text documents.

Step 4: Create a CountVectorizer object.

Step 5: Call fit_transform() on the documents to learn the vocabulary and to count the occurrences of every word in every document.

Step 6: Obtain the vocabulary using get_feature_names_out() and print its size and the matrix shape.

Step 7: Convert the sparse matrix into a pandas DataFrame using toarray().

Step 8: Print the transposed DataFrame so that rows are words and columns are documents.

Step 9: Stop the program.

CODE:

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

docs = ["NLP makes computers understand language",
        "Computers learn language using NLP",
        "Machine learning helps NLP learn language patterns"]

cv = CountVectorizer()
X = cv.fit_transform(docs)

print("Vocabulary size :", len(cv.get_feature_names_out()))
print("Features        :", list(cv.get_feature_names_out()))
print("Matrix shape    :", X.shape, "(documents x words)\n")

df = pd.DataFrame(X.toarray(), columns=cv.get_feature_names_out(),
                  index=["Doc1", "Doc2", "Doc3"])
print("Bag of Words matrix (rows = words, columns = documents):")
print(df.T)
```

✓0s

OUTPUT:

```
Vocabulary size : 11
Features        : ['computers', 'helps', 'language', 'learn', 'learning', 'machine', 'makes',
    'nlp', 'patterns', 'understand', 'using']
Matrix shape    : (3, 11) (documents x words)

Bag of Words matrix (rows = words, columns = documents):
            Doc1  Doc2  Doc3
computers      1     1     0
helps          0     0     1
language       1     1     1
learn          0     1     1
learning       0     0     1
machine        0     0     1
makes          1     0     0
nlp            1     1     1
patterns       0     0     1
understand     1     0     0
using          0     1     0
```

RESULT:

The three documents produced a vocabulary of 11 words and a document-term matrix of shape (3, 11). The words "language" and "nlp" occurred once in every document, "learn" occurred in Doc2 and Doc3, and "makes" occurred only in Doc1. Thus the text was converted into numerical vectors using the Bag of Words model; it ignores word order and treats "learn" and "learning" as different words.

EXPERIMENT NO: 9

TF-IDF (TERM FREQUENCY - INVERSE DOCUMENT FREQUENCY)

Date: _____

AIM:

To compute the TF-IDF weights of the words of a set of documents using TfidfVectorizer, and to find the similarity between the documents using cosine similarity.

ALGORITHM:

Step 1: Start the program.

Step 2: Import pandas, TfidfVectorizer and cosine_similarity from scikit-learn.

Step 3: Create a list of three documents and their names.

Step 4: Create a TfidfVectorizer object and call fit_transform() on the documents.

Step 5: Calculate the term frequency TF = (count of the word in the document) / (total words in the document) and the inverse document frequency IDF, which is computed by the vectorizer; the product is the TF-IDF weight (normalized to unit length).

Step 6: Print the TF-IDF matrix as a DataFrame, with words as rows and documents as columns.

Step 7: Print the IDF value of every word in ascending order.

Step 8: Calculate the cosine similarity between every pair of documents and print the similarity matrix.

Step 9: Stop the program.

CODE:

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = ["NLP makes computers understand language",
        "Computers learn language using NLP",
        "Machine learning helps NLP learn language patterns"]
names = ["Doc1", "Doc2", "Doc3"]

tfidf = TfidfVectorizer()
X = tfidf.fit_transform(docs)
words = tfidf.get_feature_names_out()

print("TF-IDF matrix (rows = words, columns = documents):")
print(pd.DataFrame(X.toarray().round(3), columns=words, index=names).T)

print("\nIDF of each word (rare words get higher IDF):")
for w, v in sorted(zip(words, tfidf.idf_), key=lambda p: p[1]):
    print(f"  {w:<12}{v:.3f}")

print("\nCosine similarity between documents:")
print(pd.DataFrame(cosine_similarity(X).round(3), index=names, columns=names))
```

✓0s

OUTPUT:

```
TF-IDF matrix (rows = words, columns = documents):
             Doc1   Doc2   Doc3
computers   0.420  0.450  0.000
helps       0.000  0.000  0.435
language    0.326  0.350  0.257
learn       0.000  0.450  0.331
learning    0.000  0.000  0.435
machine     0.000  0.000  0.435
makes       0.552  0.000  0.000
nlp         0.326  0.350  0.257
patterns    0.000  0.000  0.435
understand  0.552  0.000  0.000
using       0.000  0.592  0.000

IDF of each word (rare words get higher IDF):
  language    1.000
  nlp         1.000
  computers   1.288
  learn       1.288
  helps       1.693
  learning    1.693
  machine     1.693
  makes       1.693
  patterns    1.693
  understand  1.693
  using       1.693

Cosine similarity between documents:
       Doc1   Doc2   Doc3
Doc1  1.000  0.417  0.168
Doc2  0.417  1.000  0.329
Doc3  0.168  0.329  1.000
```

RESULT:

Words that occur in all documents ("language" and "nlp") received the lowest IDF of 1.000, while words occurring in only one document (such as "machine", "makes" and "using")

received the highest IDF of 1.693, so rare words get higher weights. The cosine similarity showed that Doc1 and Doc2 are the most similar (0.417) and Doc1 and Doc3 are the least similar (0.168). Thus TF-IDF weights were computed successfully.

EXPERIMENT NO: 10

SENTIMENT ANALYSIS

Date: _____

AIM:

To classify the sentiment of customer reviews as positive, negative or neutral using the lexicon-based VADER (Valence Aware Dictionary and sEntiment Reasoner) sentiment analyzer.

ALGORITHM:

Step 1: Start the program.

Step 2: Import SentimentIntensityAnalyzer from vaderSentiment.vaderSentiment and create an analyzer object.

Step 3: Create a list of reviews expressing different opinions.

Step 4: For every review, call polarity_scores() to obtain the negative, neutral, positive and compound scores.

Step 5: If the compound score is greater than or equal to 0.05, label the review as Positive.

Step 6: If the compound score is less than or equal to -0.05, label the review as Negative; otherwise label it as Neutral.

Step 7: Print every review with its scores and its label.

Step 8: Stop the program.

CODE:

```python
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

sia = SentimentIntensityAnalyzer()

reviews = ["I absolutely love this phone, the camera is fantastic!",
           "The delivery was late and the product is terrible.",
           "The package arrived on Tuesday.",
           "Not bad, but the battery could be better.",
           "Worst service ever, I am very disappointed."]

for i, review in enumerate(reviews, 1):
    s = sia.polarity_scores(review)
    if s["compound"] >= 0.05:
        label = "Positive"
    elif s["compound"] <= -0.05:
        label = "Negative"
    else:
        label = "Neutral"
    print(f"Review {i}: {review}")
    print(f"   neg={s['neg']}  neu={s['neu']}  pos={s['pos']}  "
          f"compound={s['compound']}  ->  {label}\n")
```

✓0s

OUTPUT:

```
Review 1: I absolutely love this phone, the camera is fantastic!
   neg=0.0  neu=0.455  pos=0.545  compound=0.855  ->  Positive

Review 2: The delivery was late and the product is terrible.
   neg=0.279  neu=0.721  pos=0.0  compound=-0.4767  ->  Negative

Review 3: The package arrived on Tuesday.
   neg=0.0  neu=1.0  pos=0.0  compound=0.0  ->  Neutral

Review 4: Not bad, but the battery could be better.
   neg=0.0  neu=0.51  pos=0.49  compound=0.698  ->  Positive

Review 5: Worst service ever, I am very disappointed.
   neg=0.6  neu=0.4  pos=0.0  compound=-0.8173  ->  Negative
```

RESULT:

Out of five reviews, two were classified as Positive (compound 0.855 and 0.698), two as Negative (compound -0.4767 and -0.8173) and one as Neutral (compound 0.0). The mixed review "Not bad, but the battery could be better" was labelled Positive because the lexicon treats "not bad" as a favourable phrase. Thus sentiment analysis was implemented successfully using VADER.